Extending JPEG with neural networks (X-JPEG): Image-adaptive quantization with neural networks for JPEG

Migel Tissera
Trinity Cloud

Technical Report — 4 August 2026

https://github.com/trinity-cloud/x-jpeg https://huggingface.co/migtissera/x-jpeg

Abstract

JPEG is universally deployable because its bitstream and decoder are embedded in browsers, operating systems, cameras, content-delivery networks, and hardware. Its encoder, however, must decide how strongly to quantize each of 64 spatial-frequency coefficients in every color component. Conventional encoders begin with fixed, human-designed quantization-table shapes and expose a scalar quality control. We recast table selection as an image-conditioned prediction problem.

X-JPEG is an encoder-side neural extension that predicts three 8×8 quantization tables—one each for Y, Cb, and Cr—from the source image. A 2.20-million-parameter convolutional analysis transform combines a 256×256 image view with full-resolution DCT-band statistics. The predicted integer tables are passed to an ordinary JPEG encoder; the file is standards-compliant and needs no neural decoder. Training uses differentiable JPEG, exact-forward straight-through rounding, RGB MS-SSIM with a luma guard, and a learned DCT-symbol rate proxy.

On 1,000 held-out COCO val2017 images at approximately 0.8 complete-file bits per pixel (bpp), the released three-table model reaches 0.97322 RGB MS-SSIM. It exceeds its fixed-average-table control by 0.000444 (paired 95% bootstrap CI 0.000168–0.000731) and its shuffled-table control by 0.001427 (0.000997–0.001844), establishing that image/table assignment contributes to quality. A two-emitted-table path reaches 0.97345 versus 0.97282 for jpegli at nearly identical rate, while jpegli remains better on luma-only MS-SSIM. An exploratory 100-image native study improves over Annex-K tables under the same MozJPEG backend by 0.00355 RGB MS-SSIM near 0.5 bpp and 0.00527 near 0.25 bpp; WebP remains strongest in the 0.25-bpp pilot.

1 Introduction

Learned image compression has produced strong rate–distortion results, but most learned codecs replace the bitstream, the decoder, or both. That trade is difficult across the open web, where JPEG's accumulated compatibility is itself a product feature.

This work starts from a different constraint: *the decoder and file format remain ordinary JPEG*. A neural network may spend computation while encoding, but it must leave behind only information a standard decoder already understands. Quantization tables are an unusually attractive intervention point. They are explicit encoder decisions, carried in Define Quantization Table (DQT) segments, and consumed by every conforming decoder [1, 2].

Traditional quantization tables are fixed frequency-shaping priors. A quality setting normally scales those priors but does not reconsider their shape for the image. Yet portraits, foliage, illustrations, and night photographs distribute energy and perceptual importance differently across frequency and color. X-JPEG asks whether a network can predict a better table shape for every image without changing JPEG syntax.

Predicted variation alone is insufficient. A model can collapse to one global table or emit arbitrary image-dependent variation hidden by later operating-point search. We define *useful adaptivity* causally: the correctly paired image/table must beat both a dataset-average predicted table and wrong-image tables under common rate control. The released model passes both controls on 1,000 held-out images.

Our contributions are:

1. an encoder-only neural extension that predicts image-specific standard JPEG tables;
2. a compact three-table architecture combining a thumbnail with frequency-aligned full-resolution DCT statistics;
3. a JPEG-aware training graph with straight-through quantization, perceptual distortion, learned rate estimation, and optional table-entropy/autoencoder regularizers;
4. a complete-file evaluation with fixed-average and shuffled-table adaptivity controls; and
5. open source, released weights, pinned backends, and cross-platform Python wheels.

This report concerns the independently trained 2026 release. It does not report results from or distribute weights belonging to the earlier private implementation that motivated the reconstruction.

2 JPEG as an optimization problem

2.1 Baseline pipeline

JPEG commonly transforms RGB into one luma and two chroma components, subsamples chroma, level-shifts samples, and divides each component into non-overlapping 8×8 blocks [1, 2]. For a block B, the two-dimensional DCT is

$$F = DBD^{\top}, \tag{1}$$

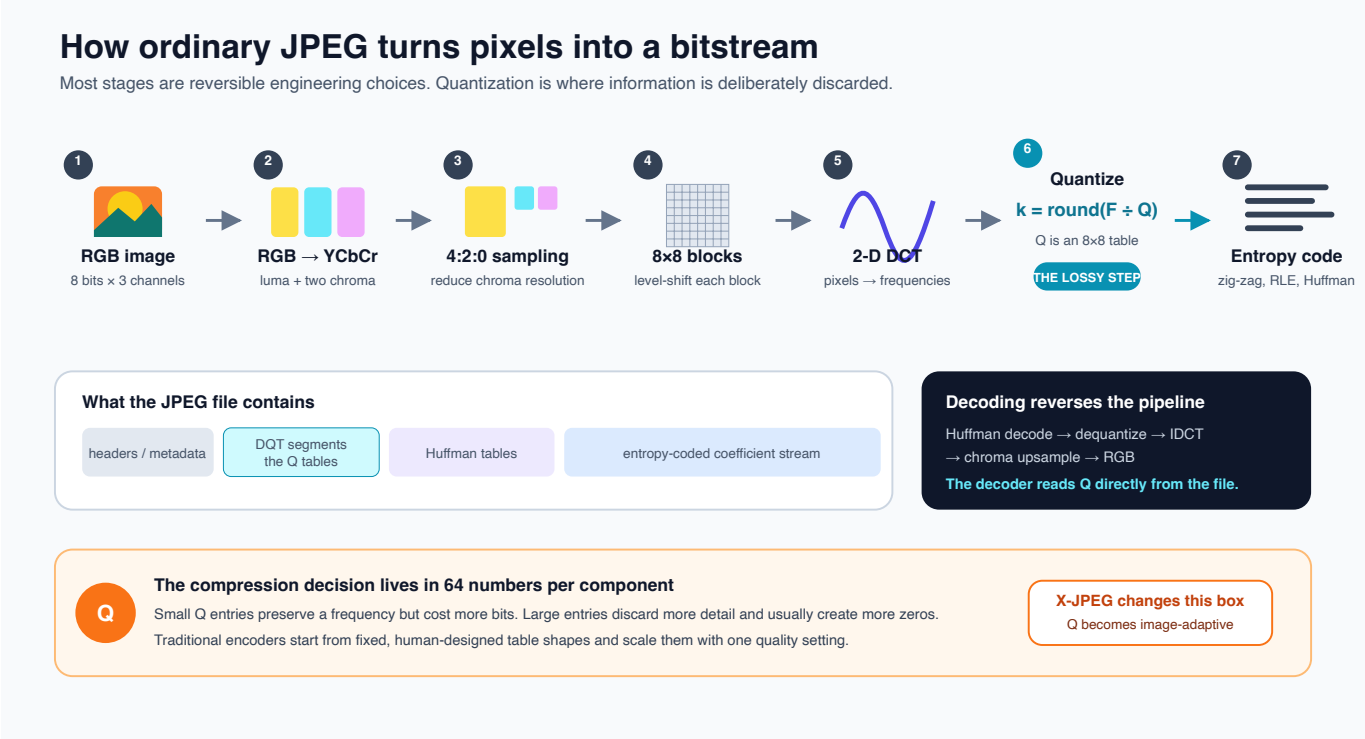


Figure 1: Baseline JPEG encoding. Quantization is the central lossy decision. Q tables are written into the file, so changing how an encoder selects them does not require a new decoder.

where D is the orthonormal eight-point DCT matrix. Given component table Q, the encoder transmits

$$K_{u,v} = \mathrm{round}(F_{u,v}/Q_{u,v})\,. \tag{2}$$

The decoder reconstructs $\hat{F} = K \odot Q$, applies the inverse DCT, upsamples chroma, and converts to RGB. Zig-zag ordering, run-length coding, DC prediction, and Huffman coding compress the symbols without additional distortion.

Small Q entries preserve a frequency more accurately but usually require more bits; large entries discard more information and produce more zeros. Because JPEG transmits Q, the tables are encoder policy rather than immutable decoder behavior (Fig. 1).

2.2 Scope of adaptivity

X-JPEG predicts one table set per image. The tables are global within each image and component. This is image-adaptive, not block-adaptive: all luma blocks share the Y table. This choice preserves ordinary JPEG syntax but cannot perform the spatial allocation available to formats with segmentation or block-level controls, such as lossy WebP [3].

2.3 Rate definition

Every reported rate is computed from the complete file,

$$\mathrm{bpp} = \frac{8\,|\text{encoded file in bytes}|}{HW}. \tag{3}$$

The numerator includes headers, DQT payloads, Huffman tables, scan headers, and entropy-coded data. We do not divide by three color channels. Thus 0.8 bpp means 0.8 compressed bits per RGB pixel location, compared with 24 uncompressed bits for 8-bit RGB.

3 Related work

Ballé, Laparra, and Simoncelli introduced end-to-end learned analysis/synthesis transforms with generalized divisive normalization (GDN) and continuous quantization relaxations [4, 5]. Hyperpriors later improved entropy estimates through learned side information [6]. These systems established the power of learned nonlinear transforms, but generally require a custom decoder and bitstream.

Straight-through estimators provide gradient surrogates for hard operations [7]. Differentiable JPEG implementations approximate the codec; Reich et al. show that color conversion, subsampling, rounding, and boundary conventions materially affect fidelity to a real implementation [8]. We use a differentiable training model but re-encode every reported point with a real codec.

DeepN-JPEG and related table optimization target neural-network classification rather than human-perceptual reconstruction [9, 10]. Neural JPEG jointly learns frequency-domain pre/post-editing and quantization tables, placing learned processing at both encoder and decoder [11]. X-JPEG occupies a narrower compatibility point: deployed neural computation selects only standard DQT values. There is no learned prefilter a decoder must reverse and no learned post-processing after decode.

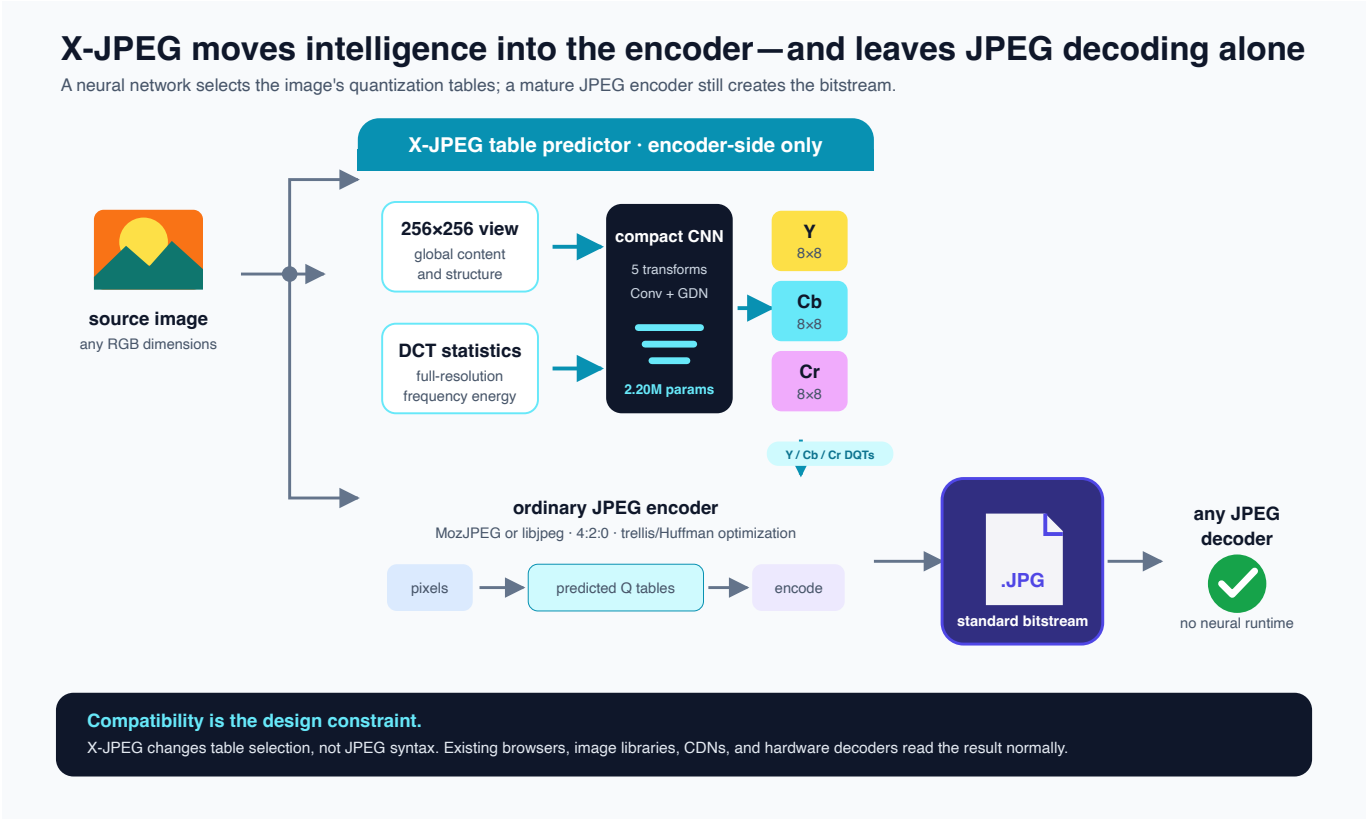


Figure 2: X-JPEG deployment boundary. Source pixels and predicted tables enter a mature JPEG encoder. The output is a normal JPEG; no model is required to decode it.

4 Method

4.1 Image-conditioned table prediction

For RGB image x, the network predicts

$$T_\theta(x) = \left(Q^Y, Q^{Cb}, Q^{Cr}\right), \quad Q^c \in \{1, \ldots, 255\}^{8\times 8}. \quad (4)$$

It receives two complementary views: a bicubic 256×256 RGB thumbnail for global content, and a $3 \times 8 \times 8$ tensor of full-resolution log DCT-band energy. The latter retains frequency evidence that thumbnailing can erase.

Four 5×5, stride-2 convolution + reparameterized GDN blocks with widths 64, 128, 192, and 256 map spatial dimensions $256 \to 128 \to 64 \to 32 \to 16$. A final 5×5, stride-2 convolution emits $3 \times 8 \times 8$ raw codes. Each full-resolution statistic is added to the matching component/frequency code with a learned scalar gain, avoiding convolution across unrelated DCT bands.

Raw codes map to legal table values via

$$T(z) = 1 + 254\,\sigma(z/4). \quad (5)$$

Training can add $\mathcal{U}(-\frac{1}{2}, \frac{1}{2})$ noise before the table-entropy model; inference rounds and clamps to $[1, 255]$. The released encoder contains 2,196,355 parameters.

4.2 Three predictions, two emission modes

The network predicts independent Y, Cb, and Cr tables. JPEG can emit all three. For small files, however, a third DQT costs roughly 65 bytes. The default product path averages the two predicted chroma tables at the encoding boundary and emits Y plus shared chroma. The three-output predictor remains unchanged. Three DQTs provide the clean adaptivity experiment; two emitted DQTs often provide the better small-file product trade-off.

4.3 Differentiable JPEG

The training codec implements RGB $\to$ YCbCr $\to$ 4:2:0 $\to$ block DCT $\to$ table division $\to$ quantization and the inverse path. For $s = F/Q$, straight-through rounding is

$$\mathrm{STEround}(s) = s + \mathrm{stopgrad}(\mathrm{round}(s) - s). \quad (6)$$

The forward value is exact integer rounding and the surrogate backward derivative is one. This prevents the predictor from exploiting fractional signal a real JPEG would discard.

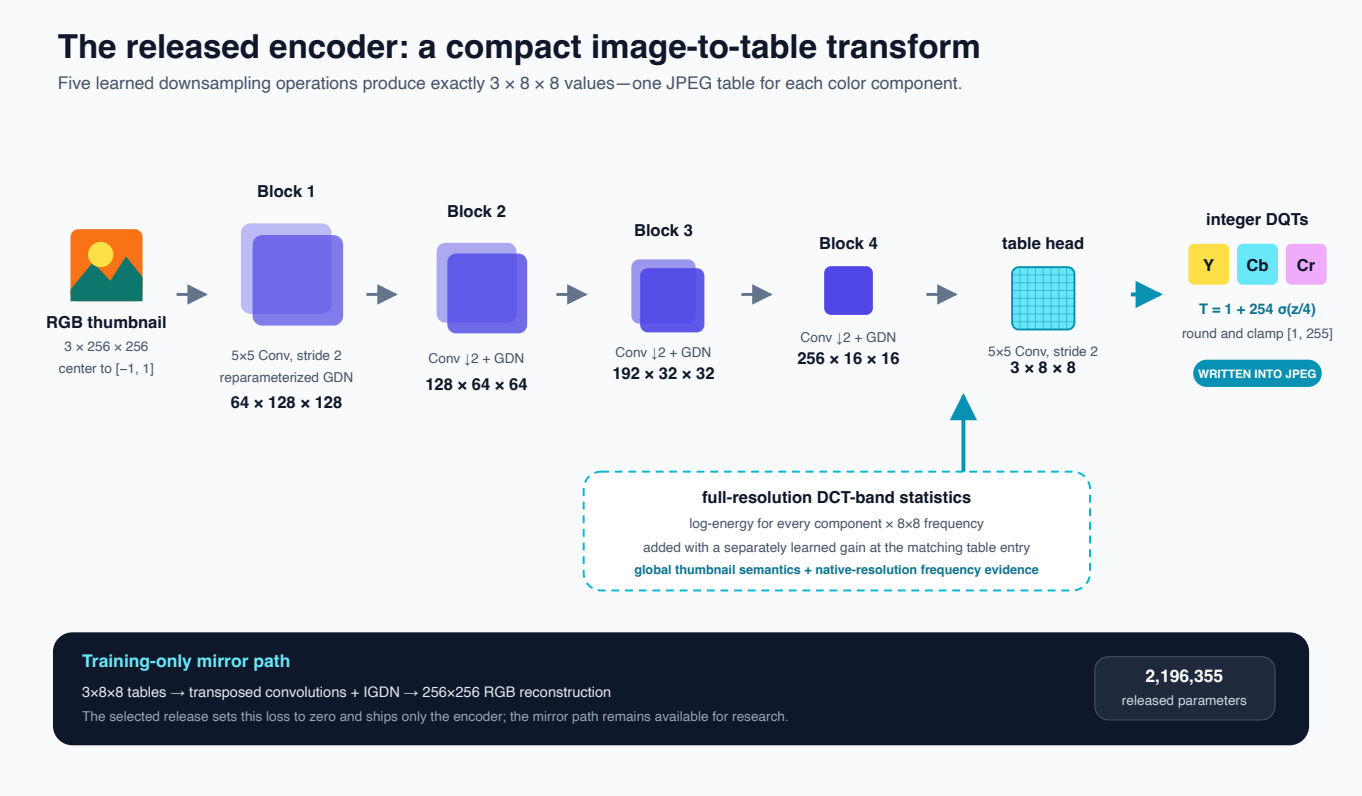


Figure 3: Released compact encoder. A mirror transposed-convolution/IGDN decoder is training-only and is absent from the inference artifact.

4.4 Objective

The training family uses

$$\mathcal{D} = 1 - \mathrm{MSSSIM}_{RGB} + \lambda_L\big(1 - \mathrm{MSSSIM}_Y\big), \tag{7}$$

$$\mathcal{L} = \lambda_d\mathcal{D} + \lambda_r R_{DCT} + \lambda_T R_{table} + \lambda_{AE}\mathcal{L}_{AE} + \lambda_{MSE}\mathcal{L}_{MSE}. \tag{8}$$

The released continuation uses $\lambda_d = 1$, $\lambda_L = 0.25$, $\lambda_r = 0.05$, and zero final weights for table entropy, mirror reconstruction, and JPEG MSE.

For each component and frequency, a learned Laplace density estimates quantized coefficient mass,

$$R_{DCT} = -\sum_i \log_2 P_\phi(k_i - \tfrac{1}{2} < K_i < k_i + \tfrac{1}{2}). \tag{9}$$

This omits exact DC differencing, run symbols, categories, optimized Huffman lengths, and headers. It is a gradient, not reported bitrate.

A Ballé-style factorized density can similarly estimate soft table entropy R_{table}. Since DQT entries are stored literally, this is a regularizer rather than a file-size saving. Ablation selected $\lambda_T = 0$. A mirror IGDN decoder can reconstruct the thumbnail from the table bottleneck, encouraging early image conditioning; it is optional, has zero weight in the selected continuation, and is not shipped.

4.5 Real encoding and rate control

At deployment, tables are multiplied by scalar s, rounded, clamped, and passed to MozJPEG or libjpeg. The public API can search s per image for a target rate or minimum RGB MS-SSIM. Scientific adaptivity tests instead use one dataset-wide scalar so rate search cannot compensate for a wrong table.

Official wheels bundle pinned MozJPEG. MozJPEG retains JPEG compatibility while adding trellis quantization, optimized Huffman coding, progressive scan optimization, and perceptual presets [12]. X-JPEG proposes table shape; MozJPEG efficiently codes coefficients under it.

5 Experiments

5.1 Training and protocol

The release model was trained on COCO train2017 [13] using random 256×256 crops. The final product lineage adds frequency statistics and a 0.25 luma guard in a 2,500-step stage followed by a 5,000-step continuation (batch 96, model learning rate 10^{-4}, rate-model learning rate 10^{-3}), initialized from an earlier RGB/rate model.

Evaluation uses held-out COCO val2017 in a controlled 256 domain and at native resolution. All encoders receive identical lossless pixels, use 4:2:0, one dataset-wide operating parameter, and a common Pillow/libjpeg decode. We report full-file bpp, canonical five-scale RGB MS-SSIM [14], luma MS-SSIM, 6:1:1 Y:Cb:Cr MS-SSIM, and RGB PSNR.

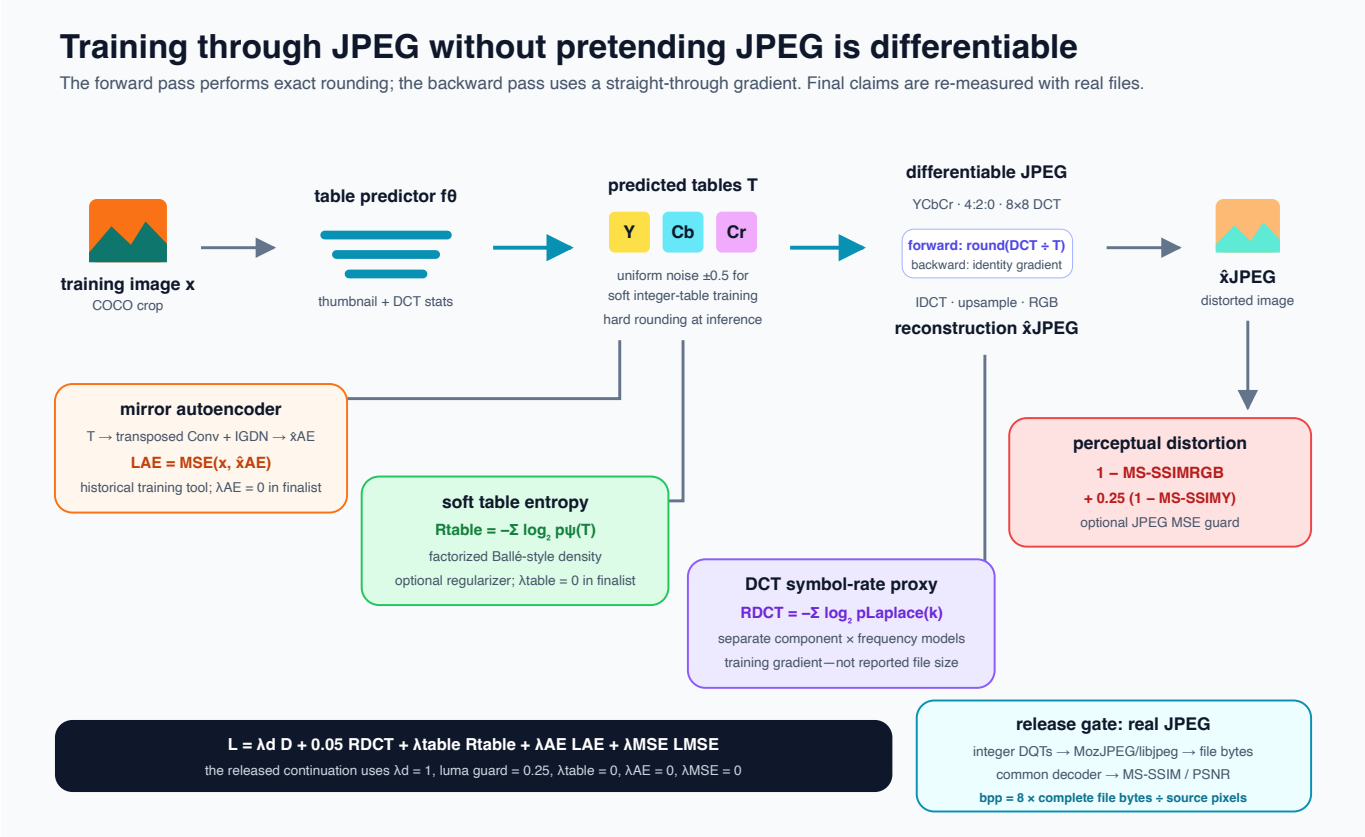


Figure 4: Training graph and release gate. Proxy losses enable optimization; a real encoder, complete file size, and common decoder determine release metrics.

The high-power study uses 1,000 images. The exploratory low-rate study uses 100 native images and compares X-JPEG+MozJPEG, Annex-K+the same MozJPEG, MozJPEG's MS-SSIM mode, and WebP method 6. MozJPEG is pinned at `0826579...1437`; jpegli at `031a007...5620` [15].

5.2 High-power result near 0.8 bpp

The shared-chroma path exceeds jpegli by 0.000628 paired RGB MS-SSIM (95% bootstrap CI 0.000389–0.000871) while using 0.00050 fewer bpp. It gains 0.00111 in 6:1:1 MS-SSIM and 0.23 dB RGB PSNR. This is not a universal quality win: jpegli is 0.00267 higher on luma-only MS-SSIM.

On native COCO-val-1000, three-DQT X-JPEG reaches 0.79981 bpp / 0.97022 RGB MS-SSIM versus jpegli at 0.79985 / 0.96892. The paired difference is 0.001299 (95% CI 0.000970–0.001635). X-JPEG gains 0.00096 in 6:1:1 and 0.11 dB RGB PSNR; jpegli is 0.00221 higher on luma.

5.3 Does the network use the image?

Across the 1,000-image set, table entries have mean standard deviation 7.923, or 21.266% relative variation. Variation alone does not prove utility. Table 2 compares correct assignment, the fixed dataset-average table, cyclically shuffled tables, and Annex-K under one scalar per method.

Adaptive minus fixed is +0.000444 RGB MS-SSIM (95% CI +0.000168 to +0.000731); adaptive minus shuffled is +0.001427 (+0.000997 to +0.001844). Both intervals exclude zero. The predictor therefore contributes more than a learned global table: matching a table to its image matters.

5.4 Exploratory low-rate behavior

At 0.5 bpp, two-DQT X-JPEG is +0.00355 RGB MS-SSIM over Annex-K under the same MozJPEG backend, +0.00050 over WebP, and −0.00041 below the tuned MozJPEG point (whose rate is 0.00643 bpp higher). At 0.25 bpp, X-JPEG is +0.00527 over Annex-K and +0.00268 over the measured MozJPEG point, but WebP is +0.00488 ahead. The gain over a fixed JPEG table grows under the tighter budget, but this 100-image aggregate motivates rather than settles the low-rate comparison.

6 Ablations

Native DCT statistics. Frequency-aligned full-image statistics improve the direct libjpeg path over equally trained no-stat controls by about +0.00054 RGB MS-SSIM at 0.8 bpp and +0.00080 at 0.5 bpp. On COCO-val-1000 near 0.5 bpp, adaptive minus fixed is +0.000564 (95% CI +0.000118–+0.001004). Learned statistic gains move materially away from initialization.

Table 1: COCO-val-1000, controlled 256 domain. Complete-file rate; best in bold.

Encoder	bpp ↓	RGB MS-SSIM ↑	Y MS-SSIM ↑	6:1:1 ↑	PSNR ↑
libjpeg-turbo / Annex-K	0.7979	0.96874	**0.98592**	0.97682	29.00
X-JPEG, three emitted DQTs	0.7985	0.97322	0.98269	0.98017	29.44
X-JPEG, shared emitted chroma	**0.7981**	**0.97345**	0.98305	**0.98047**	**29.50**
jpegli	0.7986	0.97282	0.98572	0.97935	29.26



Figure 5: Evidence summary. Adaptivity uses 1,000 images and paired bootstrap intervals. Low-rate panels use 100 images and are exploratory.

Table 2: Adaptivity controls at ≈0.8 bpp, $n = 1000$.

Method	bpp	RGB MS	Y MS	PSNR
adaptive X-JPEG	0.7985	**0.97322**	0.98269	**29.44**
fixed average	0.7984	0.97278	0.98233	29.35
shuffled	0.7987	0.97180	0.98150	29.29
Annex-K	0.7989	0.96885	**0.98594**	29.01

Soft table entropy. Keeping weight 0.01 versus turning it off gives numerically indistinguishable product results, consistent with literal DQT storage. It remains available for historical reconstruction; the finalist uses zero.

Mirror decoder and initialization. The autoencoder loss is unnecessary in the selected product continuation. A small nonzero bottleneck-head initialization and explicit wrong-image controls are more important to avoiding global-table collapse. The decoder is not shipped.

Width. A controlled two-times-width run did not improve the frontier within its budget. More parameters do not automatically produce better tables; optimization and input evidence matter.

Third-DQT overhead. Three outputs preserve the full predictor, but a third DQT can hurt small files. Averaging chroma only at emission frequently improves complete-file performance by saving roughly 65 bytes.

7 Limitations

1. The model is optimized primarily for RGB MS-SSIM with a luma guard; it can lead on RGB while trailing on luma.

Table 3: Native COCO-val-100 exploratory low-rate aggregate. The study lacks paired intervals.

	0.5-bpp target		0.25-bpp target	
Encoder	bpp	RGB MS-SSIM	bpp	RGB MS-SSIM
X-JPEG, 2 DQT + MozJPEG	0.49975	0.95718	0.25111	0.91764
X-JPEG, 3 DQT + MozJPEG	0.49917	0.95693	0.25002	0.91642
Annex-K + MozJPEG	0.50097	0.95363	0.25000	0.91237
MozJPEG MS-SSIM	0.50618	**0.95759**	0.24472	0.91495
WebP method 6	0.50015	0.95668	0.24996	**0.92252**

2. Standard DQT compatibility restricts allocation to one table per component/image, not block-level adaptation.
3. The Laplace rate proxy omits material JPEG coding details; real-file validation remains mandatory.
4. Training/evaluation are dominated by natural COCO photographs. Medical, synthetic, text-heavy, scanned, and adversarial imagery are uncharacterized.
5. The 0.5/0.25-bpp comparison has 100 images and no paired intervals.
6. On an Intel Xeon Platinum 8480+, table prediction takes 10.4 ms median / 13.0 ms p95 on a 50-image subset (median 0.273 MP); stock JPEG adds 1.5 / 2.5 ms. Other hardware requires measurement.
7. Encoding requires an 8.4 MB model and current PyTorch runtime, though decoding is model-free.
8. Per-image target-rate search performs multiple encodes and must not be confused with evidence of adaptivity.
9. Final-checkpoint seed variability remains unmeasured.

8 Reproducibility

Version 0.1.0 is MIT licensed at `https://github.com/trinity-cloud/x-jpeg`; the model is at `https://huggingface.co/migtissera/x-jpeg`; installation is `pip install xjpeg`. The output opens with an ordinary JPEG decoder.

The released Safetensors SHA-256 is

```
823a305b9cdf6d16e2296644d88a343d
fc68621fa8e3d4b3c0125772c50b92e8
```

and its source encoder checkpoint SHA-256 is

```
5878df379ff23dbfa1c1e753d0a70d2b
131e4625cc20be2b61fc882789304219.
```

The current checkpoint was trained in 2026 from the open PyTorch implementation. No prior private model weights are present in the package; every empirical claim here refers to the released artifact.

9 Conclusion

X-JPEG treats a mature file format as a platform rather than an obstacle. A network makes an encoder decision JPEG already represents: which frequencies and components deserve bits in this image. The decoder remains unchanged.

The central result is not merely a gain over a standard table. Correctly pairing the released model's table with its source beats both fixed-average and wrong-image tables on 1,000 held-out samples with paired intervals above zero. This establishes genuine image-conditioned utility inside a standard JPEG bitstream. Low-rate pilots further suggest that benefit over fixed JPEG tables grows as bits become scarce, while showing that WebP remains ahead at 0.25 bpp in the present comparison.

The next research step is a preregistered multi-dataset low-rate study with paired uncertainty, visual examples, and encode-time accounting. The immediate product result already exists: X-JPEG emits files the world can decode today.

References

[1] ITU-T. Information technology—digital compression and coding of continuous-tone still images—requirements and guidelines. Technical Report Recommendation T.81, International Telecommunication Union, 1992.

[2] Gregory K. Wallace. The JPEG still picture compression standard. *IEEE Transactions on Consumer Electronics*, 38(1):xviii–xxxiv, 1992.

[3] Google. WebP compression techniques. `https://developers.google.com/speed/webp/docs/compression`, 2026.

[4] Johannes Ballé, Valero Laparra, and Eero P. Simoncelli. Density modeling of images using a generalized normalization transformation. In *International Conference on Learning Representations*, 2016.

[5] Johannes Ballé, Valero Laparra, and Eero P. Simoncelli. End-to-end optimized image compression. In *International Conference on Learning Representations*, 2017.

[6] Johannes Ballé, David Minnen, Saurabh Singh, Sung Jin Hwang, and Nick Johnston. Variational image compression with a scale hyperprior. In *International Conference on Learning Representations*, 2018.

[7] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[8] Christoph Reich, Biplob Debnath, Deep Patel, and Srimat Chakradhar. Differentiable JPEG: The devil is in the details. In *IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 4126–4135, 2024.

[9] Zihao Liu, Tao Liu, Wujie Wen, Lei Jiang, Jie Xu, Yanzhi Wang, and Gang Quan. DeepN-JPEG: A deep neural network favorable JPEG-based image compression framework. In *Design Automation Conference*, 2018.

[10] Zhijing Li, Christopher De Sa, and Adrian Sampson. Optimizing JPEG quantization for classification networks. *arXiv preprint arXiv:2003.02874*, 2020.

[11] Ankur Mali, Alexander Ororbia, Daniel Kifer, and C. Lee Giles. Neural JPEG: End-to-end image compression leveraging a standard JPEG encoder-decoder. *arXiv preprint arXiv:2201.11795*, 2022.

[12] Mozilla. MozJPEG: Improved JPEG encoder. https://github.com/mozilla/mozjpeg, 2026. Pinned evaluation commit 08265790774cd0714832c9e675522acbe5581437.

[13] Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, and Piotr Dollár. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision*, pages 740–755, 2014.

[14] Zhou Wang, Eero P. Simoncelli, and Alan C. Bovik. Multiscale structural similarity for image quality assessment. In *Proceedings of the 37th Asilomar Conference on Signals, Systems and Computers*, volume 2, pages 1398–1402, 2003.

[15] JPEG XL Project. jpegli in the JPEG XL reference implementation. https://github.com/libjxl/libjxl, 2026. Pinned evaluation commit 031a0077f5799a6041004267fc12b956c1f52a20.